

College Alumni Networking & Mentorship Platform

Project Report

Submitted by:

Aman Kumar - AP24110010423

Prem Sai - AP24110010403

Sujith Parimi - AP24110010381

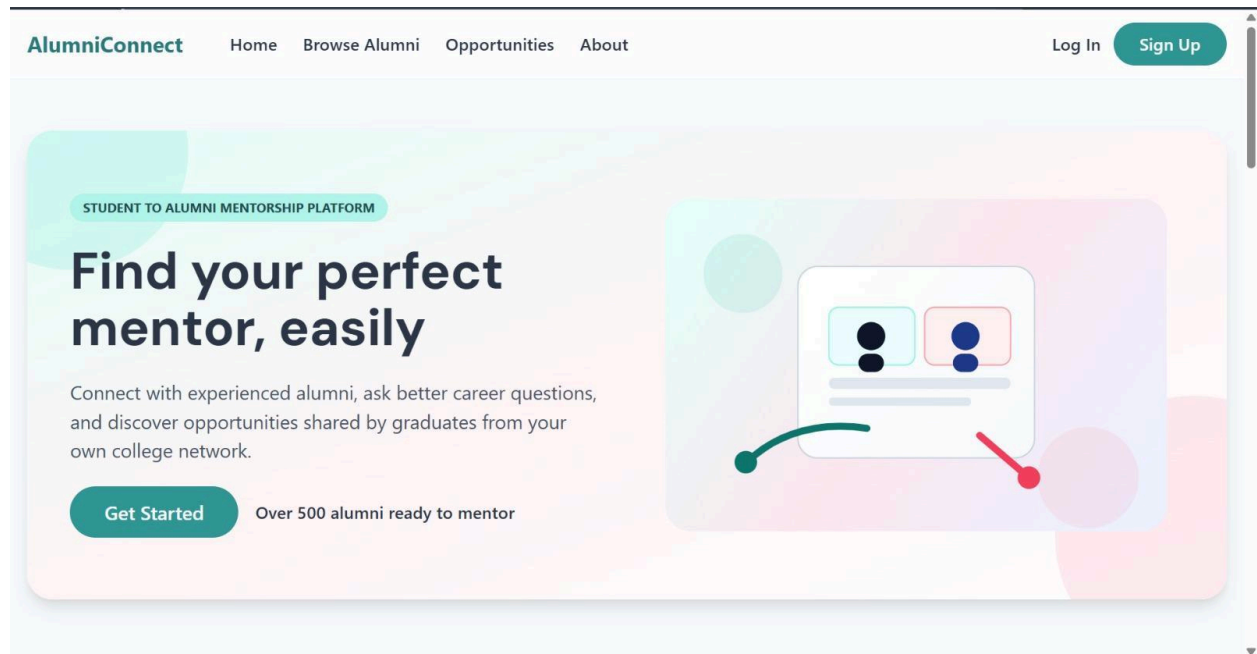
Vidyasagar Adapa - AP24110010449

Nikhil Rajendra - AP24110010512



1. Introduction

The College Alumni Networking & Mentorship Platform is a web-based system designed to connect students with alumni for mentorship, networking, and career guidance. It provides a centralized platform where users can interact, share opportunities, and build professional relationships.



The system improves communication between students and alumni, helping students gain industry insights and career support in an organized way.

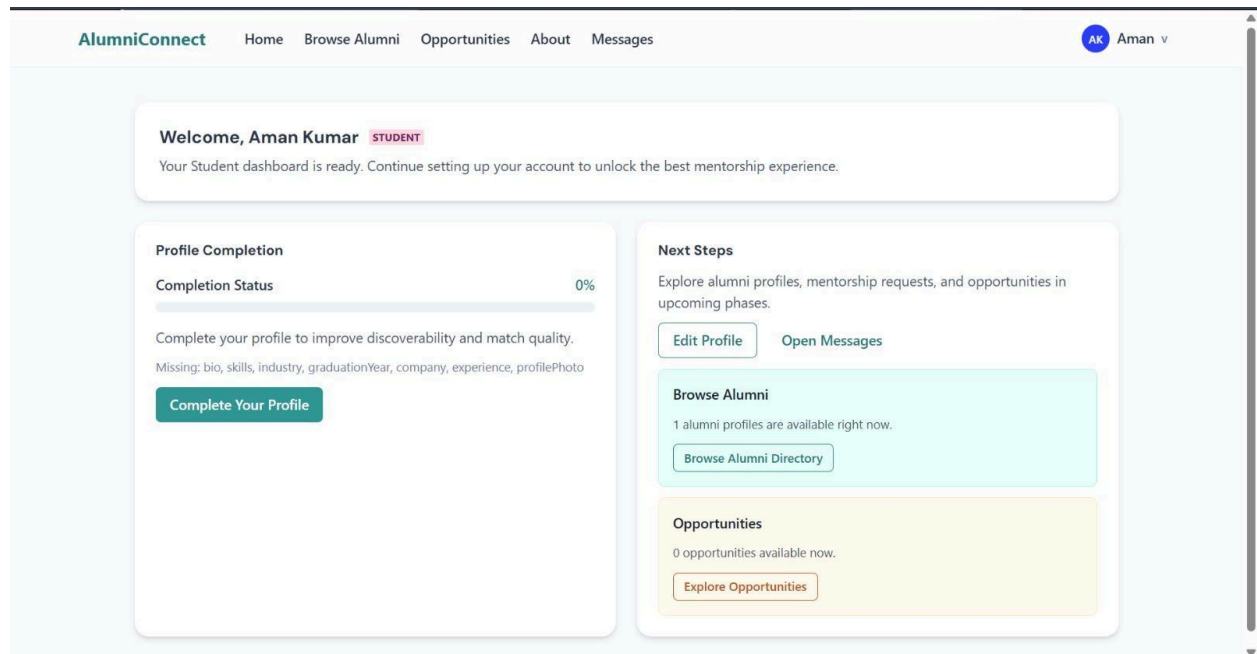
2. Description

The College Alumni Networking & Mentorship Platform is a full-stack web application that enables students and alumni to interact through a structured networking environment. The platform allows alumni to create profiles that include their professional background, industry experience, and areas of expertise. Students can browse these profiles and connect with alumni who match their career interests.

The system allows users to:

- Create and manage profiles
- Explore alumni networks
- Send mentorship requests

- Share job or internship opportunities
- Communicate with mentors
- Track mentorship activities



The platform supports role-based access for:

- Student
- Alumni
- Administrator

3. Problem Statement

In many colleges, students face challenges when trying to connect with alumni who can guide them in their career paths.

Some of the major issues include:

- Lack of structured communication between students and alumni
- Difficulty finding alumni working in specific industries
- Limited access to mentorship opportunities
- No centralized platform for sharing job or internship openings
- Unorganized alumni networks managed through social media groups

Due to these limitations, students often miss opportunities to learn from alumni experiences and build professional connections. Therefore, there is a need for a centralized digital platform that connects students and alumni, enabling mentorship, networking, and career guidance.

4. Scenario Based Case Study

Consider a college where alumni are working in various industries such as software development, data science, finance, and marketing.

Meet Arjun, a third-year Computer Science student at SRM University AP who wants career guidance in software engineering. Arjun is responsible for choosing the right specialization and building his professional network before graduating.

Arjun discovers the College Alumni Networking & Mentorship Platform, a comprehensive web application designed to streamline student-alumni connections. Intrigued by its features, Arjun decides to explore how the platform can help him enhance his career prospects.

User Registration and Authentication:

Arjun registers an account on the platform, providing his academic details and career interests. He logs in securely using JWT authentication, ensuring that his data remains private and secure.

Alumni Discovery:

The platform offers a searchable alumni directory that allows Arjun to filter alumni by industry, skills, and experience. He can easily access alumni profiles to identify the right mentor for his career path.

Mentorship Request:

Arjun sends a mentorship request to a senior software engineer alumnus. The alumni accepts and provides guidance on career paths, internships, interview preparation, and industry trends.

Opportunity Sharing:

Alumni can post job openings or internship opportunities, allowing students like Arjun to apply directly through the platform.

Thanks to the platform, Arjun can now manage his networking activities more efficiently. He focuses on building strong relationships with industry professionals, exploring opportunities, and accelerating his career growth. The platform has become an indispensable tool in his student journey.

5. System Requirements

To ensure smooth development, deployment, and usage of the Alumni Networking Platform, certain system prerequisites must be met.

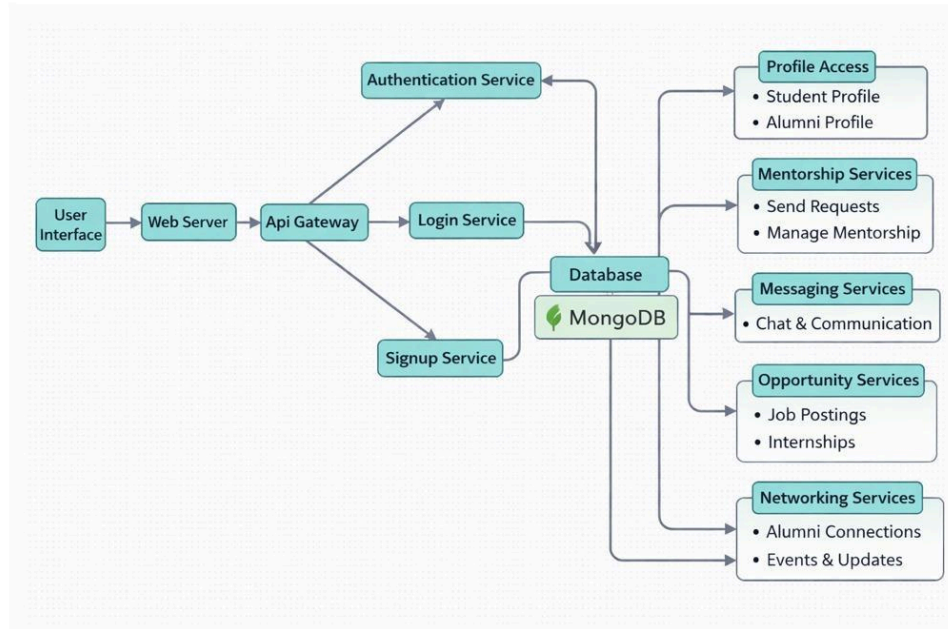
Software Requirements

Component	Specification	Purpose
Operating System	Windows 10/11, macOS, or Linux	Cross-platform development and testing
Node.js	v16 or above	Server-side runtime and frontend tooling
npm	v8 or above	Package management for dependencies
React.js	Latest stable	Dynamic and responsive UI development
Express.js	Latest stable	Lightweight REST API framework
MongoDB	v5 or above	NoSQL database for storing platform data
Browser	Chrome / Firefox (latest)	UI rendering and testing
VS Code	Latest	Code editor with Git and terminal support
Postman	Latest	API testing during development

Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 (8th Gen)	Intel Core i5 or AMD Ryzen 5
RAM	8 GB	16 GB
Storage	1 GB free space	5 GB free space
Display	1366x768	1920x1080 or higher

6. Architecture



The College Alumni Networking & Mentorship Platform follows a full-stack web application architecture built using the MERN stack (MongoDB, Express.js, React.js, Node.js).

Frontend Layer

- Built using React.js
- Provides interactive dashboards for students, alumni, and administrators
- Users can create profiles, browse alumni networks, send mentorship requests, and view job opportunities

Backend Layer

- Built using Node.js and Express.js
- Handles API requests and application logic
- Responsible for user management, mentorship request processing, job opportunity management, and messaging functionality

Database Layer

- Uses MongoDB
- Stores platform data including user profiles, mentorship requests, messages, and job opportunities

Authentication Layer

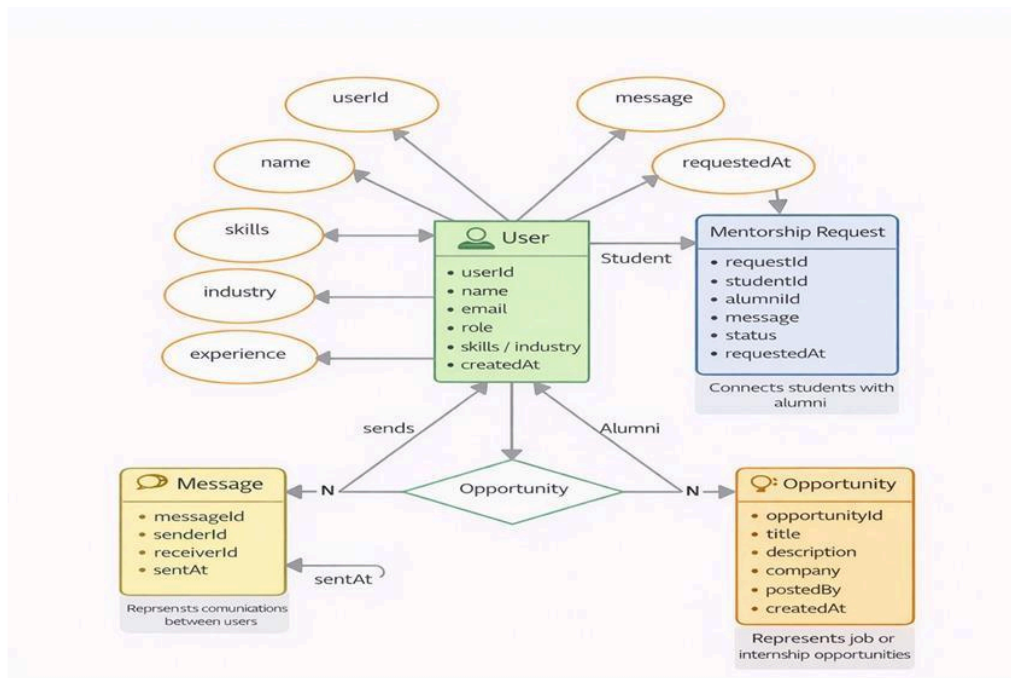
- Uses JWT (JSON Web Token) Authentication
- Provides secure login and role-based access control

Communication Layer

Handles interaction between students and alumni through:

- Messaging system
- Mentorship requests
- Networking updates and event notifications

7. ER Diagram (Entity Relationship)



The College Alumni Networking & Mentorship Platform uses an Entity-Relationship (ER) model to represent how users, mentorship requests, opportunities, and communication data are structured and related.

1. User Entity

This entity stores information about all users including students, alumni, and administrators.

Attributes include:

- userId
- name
- email
- password
- role (Student / Alumni / Admin)
- skills
- industry
- experience
- createdAt

A user can send mentorship requests, receive requests, post opportunities, and communicate with others.

2. Mentorship Request Entity

This entity manages mentorship connections between students and alumni.

Attributes include:

- requestId
- studentId
- alumniId
- message
- status (Pending / Accepted / Rejected)
- requestedAt

3. Opportunity Entity

This entity stores job or internship opportunities shared by alumni.

Attributes include:

- opportunityId
- title
- description
- company
- postedBy
- createdAt

Each opportunity is posted by an alumni and can be viewed by multiple students.

4. Message Entity

This entity stores communication between students and alumni.

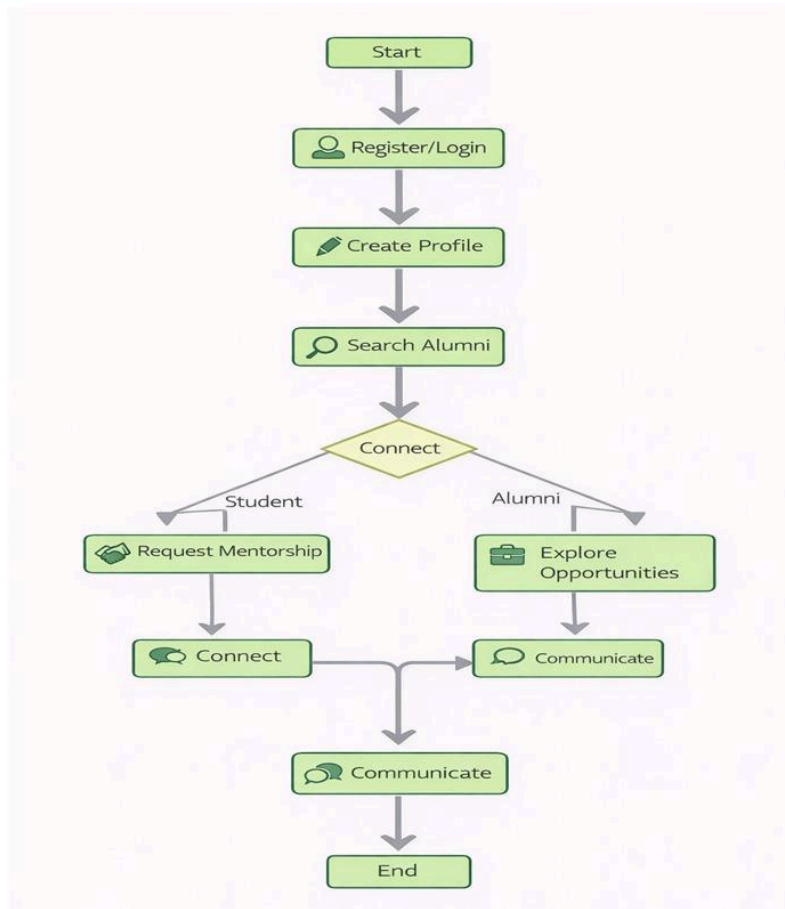
Attributes include:

- messageId
- senderId
- receiverId
- content
- sentAt

Relationships

- One User (Student) can send multiple Mentorship Requests
- One User (Alumni) can receive multiple Mentorship Requests
- One User (Alumni) can post multiple Opportunities
- One Opportunity belongs to one User (Alumni)
- One User can send and receive multiple Messages
- One Mentorship Request connects a Student and Alumni

8. Project Flow (System Flow)



Step 1: User Registration

Students and alumni register on the platform providing professional details and preferences.

AlumniConnect

[Home](#) [Browse Alumni](#) [Opportunities](#) [About](#)

[Log In](#) [Sign Up](#)

Create an Account

Join as Student or Alumni mentor

Name *

Your full name

Email *

you@example.com

Password *

Minimum 6 characters

Role *

☒ Student ☐ Alumni

Sign Up

[Already have an account? Log in](#)

Step 2: Authentication

Users log in securely using JWT authentication, ensuring data privacy and security.

AlumniConnect

[Home](#) [Browse Alumni](#) [Opportunities](#) [About](#)

[Log In](#) [Sign Up](#)

Log In

Access your AlumniConnect dashboard

Email *

you@example.com

Password *

Enter your password

Log In

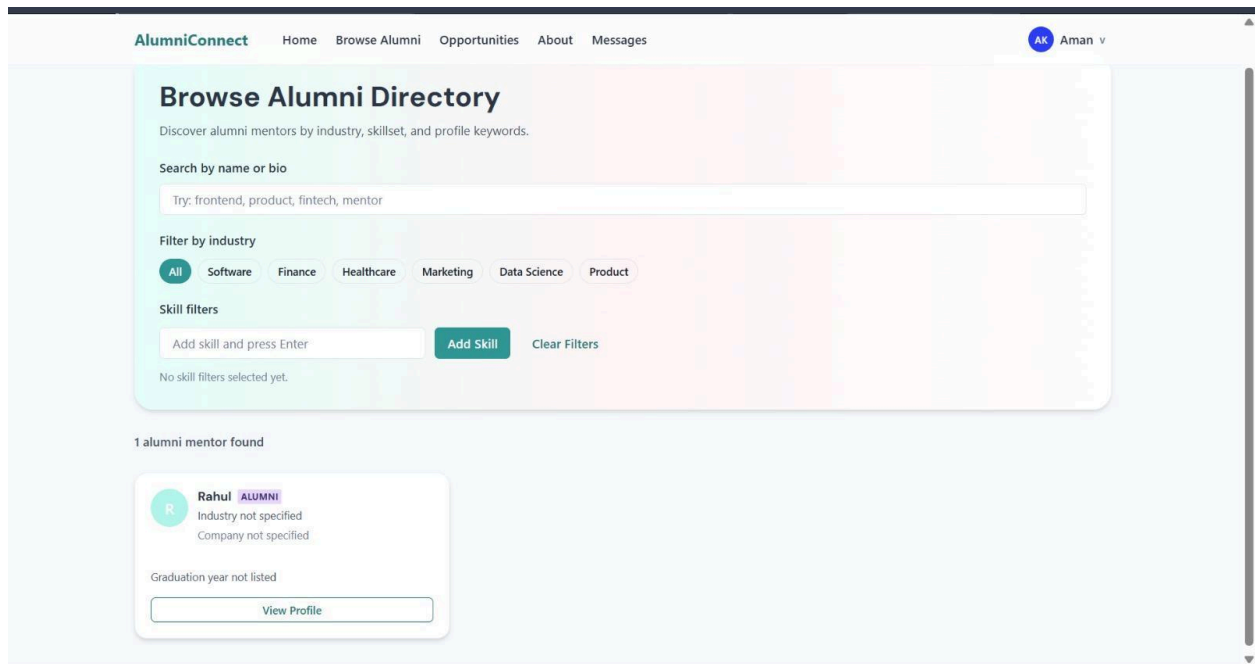
[New here? Create an account](#)

Step 3: Profile Creation

Users create profiles including skills, industry, and experience to make themselves discoverable.

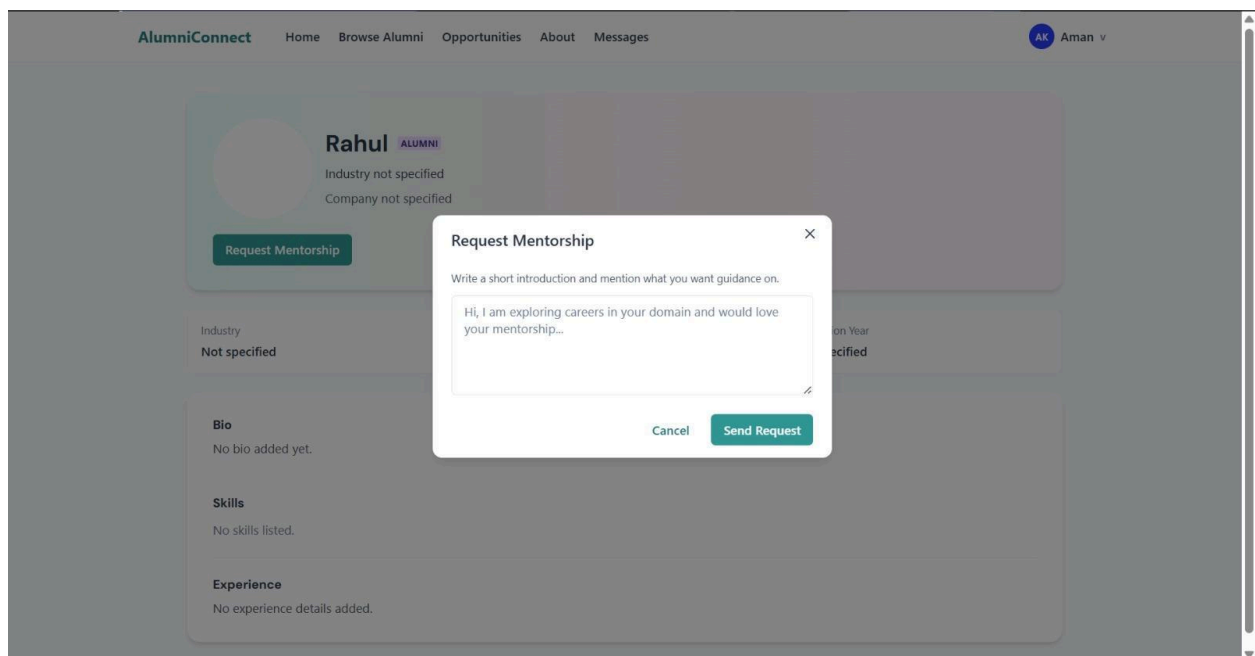
Step 4: Alumni Discovery

Students browse alumni profiles filtered by career interests, industry, and skill set.



Step 5: Mentorship Request

Students send mentorship requests to alumni professionals of their choice.

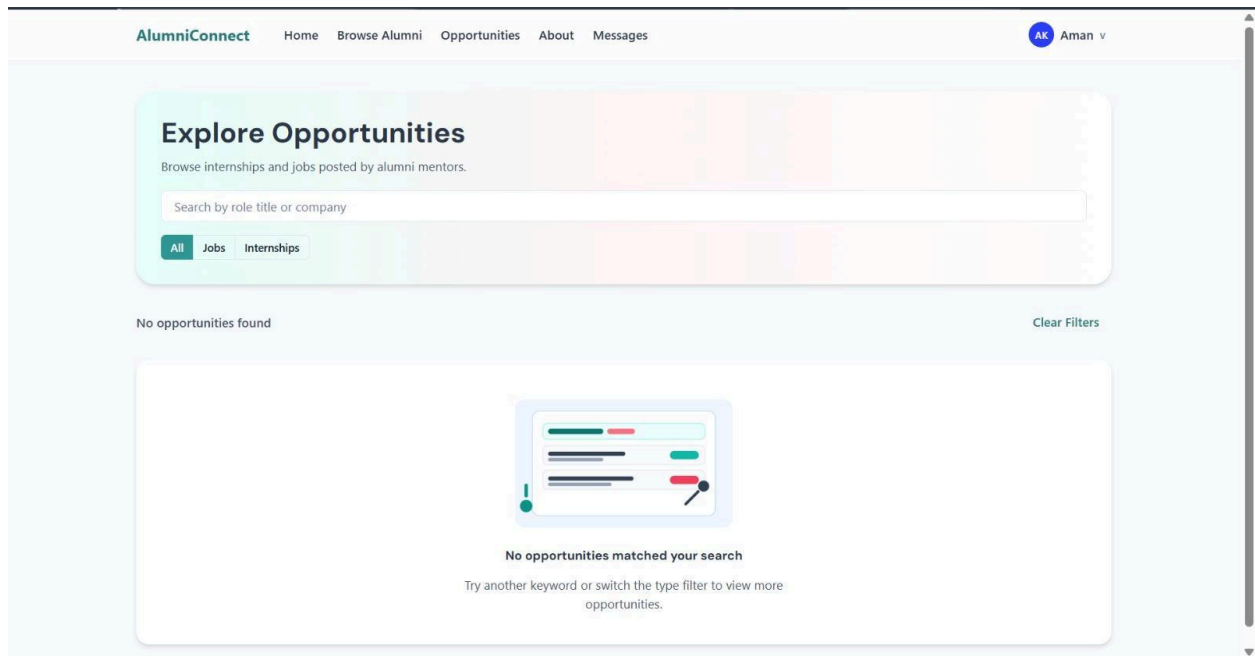


Step 6: Communication

Alumni and students communicate through the platform's built-in messaging system.

Step 7: Opportunity Sharing

Alumni share job openings or internship opportunities directly on the platform.



Step 8: Networking Activities

Users build professional connections and long-term mentorship relationships.

9. Pre-Requisites

To build this project, knowledge of the following technologies is required.

Frontend

- React Components
- React Hooks
- React Router
- Axios for API calls
- TailwindCSS / CSS

Backend

- Node.js fundamentals
- Express.js routing
- REST API development
- Middleware
- JWT Authentication

Database

- MongoDB document structure
- CRUD operations
- Mongoose ODM
- Schema design

Additional Concepts

- Role-based authorization
- Networking platform design
- Messaging systems
- Form validation

10. Required Technologies

Frontend

- React.js
- TailwindCSS / Material UI
- Axios

Backend

- Node.js
- Express.js
- JWT Authentication
- Bcrypt

Database

- MongoDB
- Mongoose

Development Tools

- Git
- GitHub
- VS Code
- Postman / Thunder Client

11. MVC Pattern

The Alumni Networking Platform follows the Model-View-Controller (MVC) architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.

Model Layer (Data Layer)

The Model layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database using those schemas. The models are implemented using Mongoose, which provides a schema-based solution to model application data for MongoDB.

Controller Layer

The Controller layer acts as an intermediary between the view (routes) and the model. It receives incoming requests, processes the input (which may include validation or transformation), calls the appropriate methods from the model, and then returns a response to the client.

View Layer (Routing Layer)

In the context of a backend REST API, the View is implemented as the routing layer, where various endpoints are defined. These endpoints determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions.

Advantages of Using MVC in This Project

- Separation of Concerns: Each layer has a clearly defined responsibility, improving readability and maintainability
- Scalability: New features can be added easily by creating new routes, controllers, and models
- Reusability: Logic in controllers and models can be reused across multiple parts of the application
- Testing: Each layer can be tested independently, especially the controllers and models
- Collaboration-Friendly: Multiple developers can work simultaneously on different layers without conflict

12. Suggested Database Collections

Users

Field	Type	Description
_id	ObjectId	Auto-generated unique identifier
name	String	Full name of the user
email	String	Email address (unique)
password	String	Bcrypt hashed password
role	String	Student / Alumni / Admin
skills	Array	List of skills
industry	String	Industry of work or interest
experience	String	Years of experience
createdAt	Date	Account creation timestamp

MentorshipRequests

Field	Type	Description
_id	ObjectId	Auto-generated unique identifier
studentId	ObjectId	Reference to User (Student)
alumniId	ObjectId	Reference to User (Alumni)
message	String	Message from student
status	String	Pending / Accepted / Rejected

requestedAt	Date	Request timestamp
-------------	------	-------------------

Opportunities

Field	Type	Description
_id	ObjectId	Auto-generated unique identifier
title	String	Job or internship title
description	String	Detailed description
company	String	Company name
postedBy	ObjectId	Reference to User (Alumni)
createdAt	Date	Post creation timestamp

Messages

Field	Type	Description
_id	ObjectId	Auto-generated unique identifier
senderId	ObjectId	Reference to User (Sender)
receiverId	ObjectId	Reference to User (Receiver)
content	String	Message content
sentAt	Date	Message timestamp

13. Project Setup and Configuration

Creating Project Folder

1. Create a new folder with your project name
2. Inside that folder, create two new folders
3. Name one as Client
4. Name another one as Server
5. Open that folder in VS Code

Client Setup (Installing React App)

Open the Client folder in the terminal of VS Code:

```
npm create vite@latest . -- --template react
```

Select React framework from the given options.

Select JavaScript variant from the given options.

Navigate to the client folder:

```
cd client
```

Install all packages:

```
npm install
```

Start the React server:

```
npm run dev
```

Server Setup (npm init)

Open Server folder in terminal of VS Code:

```
npm init -y
```

Create files:

- server.js

Create folders:

- models
- controllers
- routes

14. Backend Development

Backend Structure

Controllers

- alumniController.js - Handles all alumni-related business logic (login, manage profile, accept requests, etc.)
- studentController.js - Handles all student-related operations (register, login, send requests, explore opportunities, etc.)
- adminController.js - Handles admin operations (manage users, moderate content, etc.)

Middleware

- authMiddleware.js - Middleware for authentication/authorization (JWT validation)
- upload.js - Middleware for handling file uploads (e.g., profile pictures)

Models

- User.js - Schema/model for all user accounts (students, alumni, admins)
- MentorshipRequest.js - Schema/model for mentorship connections
- Opportunity.js - Schema/model for job and internship postings
- Message.js - Schema/model for user communications

Routes

- studentRoutes.js - Defines API routes for student actions (register, login, send request, explore, etc.)
- alumniRoutes.js - Defines API routes for alumni actions (login, accept requests, post opportunities, etc.)
- adminRoutes.js - Defines API routes for admin actions (manage users, moderate content, etc.)

15. Database Development

1. Configure MongoDB

- Install Mongoose. In your Node.js project directory, run:

```
npm install mongoose
```

- Create database connection
- Create Schemas and Models

Create Database Connection

Make sure the database is connected before performing any backend actions. The connection code uses Mongoose to connect to MongoDB via an environment variable for the URI, with proper error handling and process exit on failure.

Create Schemas

Configure the Schemas for the MongoDB database using data from the ER diagrams. The main schemas for this application are:

6. Users.js - Schema/model for all user accounts (students, alumni, admins)
7. MentorshipRequest.js - Schema/model for mentorship connections between students and alumni
8. Opportunity.js - Schema/model for job and internship postings by alumni
9. Message.js - Schema/model for user communications

16. Frontend Development

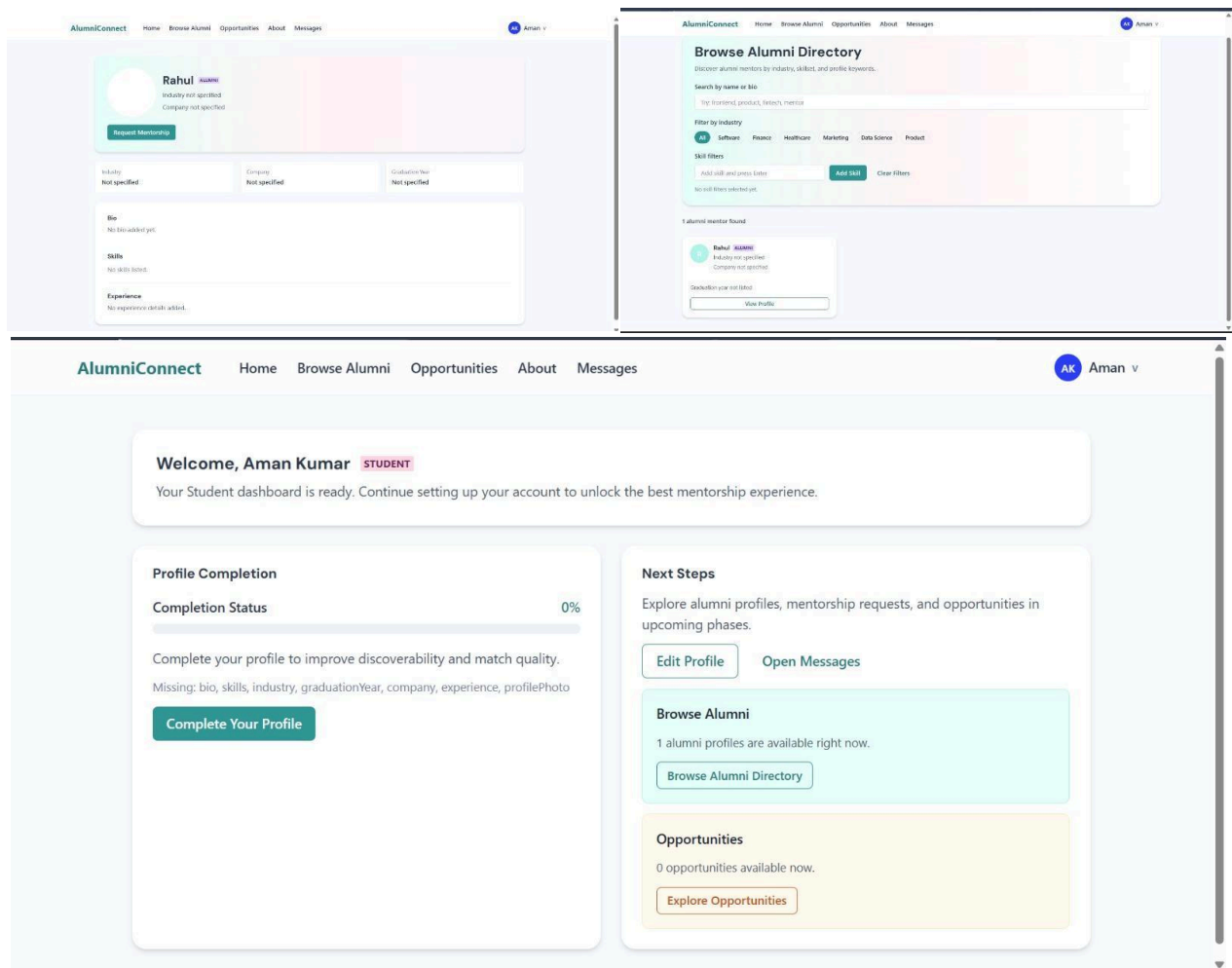
Frontend Structure

Admin (src/Admin)

- Dashboard.jsx - Admin dashboard/homepage with platform statistics
- Alogin.jsx - Admin login page
- Asignup.jsx - Admin signup/registration page
- Anavbar.jsx - Navigation bar for admin panel
- ManageUsers.jsx - Admin panel to manage registered students and alumni
- ManageRequests.jsx - Admin view for monitoring mentorship requests

Student (src/Student)

- Uhome.jsx - Student homepage/dashboard
- Ulogin.jsx - Student login page
- Usignup.jsx - Student signup/registration page
- Unavbar.jsx - Navigation bar for student side
- AlumniList.jsx - Browse and search alumni profiles
- AlumniProfile.jsx - View individual alumni profile
- MentorshipRequest.jsx - Send and track mentorship requests
- Opportunities.jsx - View job and internship listings



Alumni (src/Alumni)

- Ahome.jsx - Alumni homepage/dashboard
- AProfile.jsx - Alumni profile management
- Requests.jsx - View and respond to incoming mentorship requests
- PostOpportunity.jsx - Post new job or internship opportunities
- Messaging.jsx - Communicate with connected students

Components (Reusable)

- Footer.jsx - Footer component used across pages
- Home.jsx - Main landing page for the platform
- Navbar.jsx - Shared navigation component
- ui/ - Shared UI components (buttons, modals, cards, etc.)

17. Roles and Responsibilities

Student

- **Registration and Account Management:** Students are responsible for creating an account, providing accurate academic information, and managing their profile.
- **Alumni Discovery:** Students can search for available alumni based on their preferences such as industry, skills, and experience.
- **Mentorship Requests:** Students are responsible for sending mentorship requests and communicating with their mentors.
- **Opportunity Exploration:** Students can browse and apply for job and internship opportunities shared by alumni.
- **Feedback:** Students have the responsibility to engage constructively and provide feedback on mentorship sessions.

Alumni

- **Profile Management:** Alumni create and maintain detailed profiles showcasing their professional background and areas of expertise.
- **Mentorship:** Alumni can accept or reject mentorship requests and guide students through career planning.
- **Opportunity Sharing:** Alumni can post job openings and internship opportunities for students.
- **Communication:** Alumni communicate with mentees through the platform's messaging system.

Admin

- **System Management:** The admin is responsible for overall management and administration of the platform, including monitoring system performance and managing configurations.
- **User Management:** The admin is responsible for managing user accounts, including registration verification and account moderation.
- **Content Moderation:** The admin oversees the quality of postings and ensures platform policies are adhered to.

18. Key Features

Authentication

- User signup and login
- JWT-based secure authentication
- Role-based access control

Profile Management

- Create and update professional profiles
- Showcase skills, industry, and experience

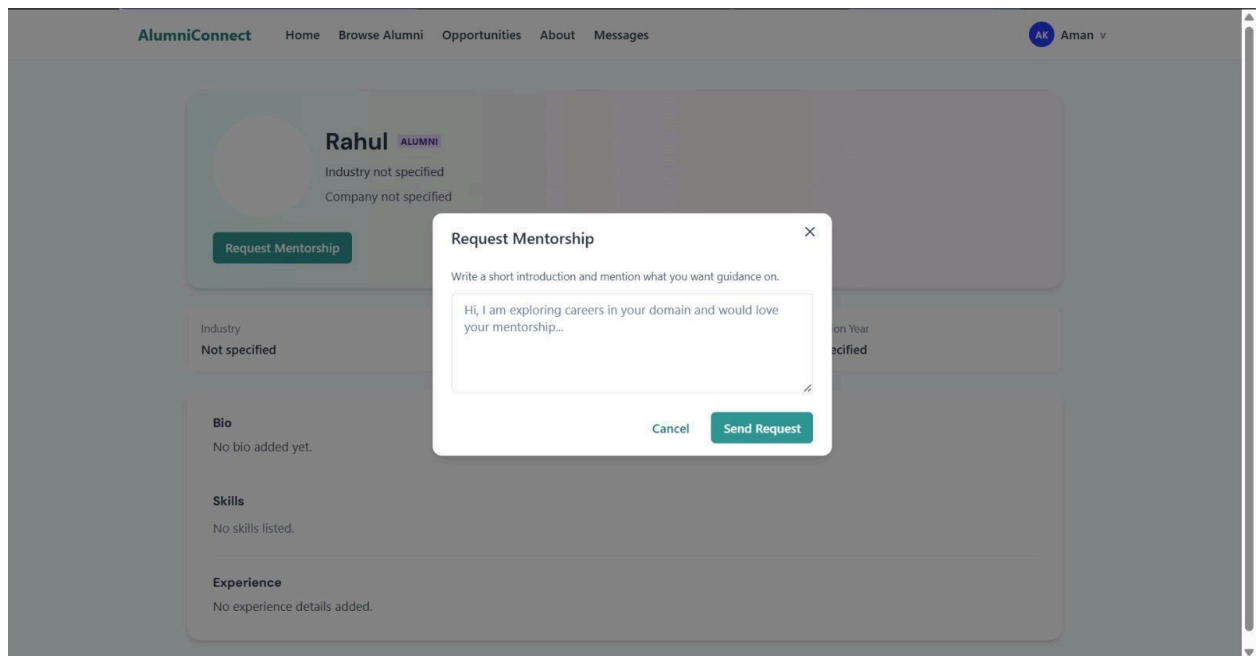
Alumni Discovery

- Search alumni by industry and skill
- View detailed alumni profiles

Mentorship System

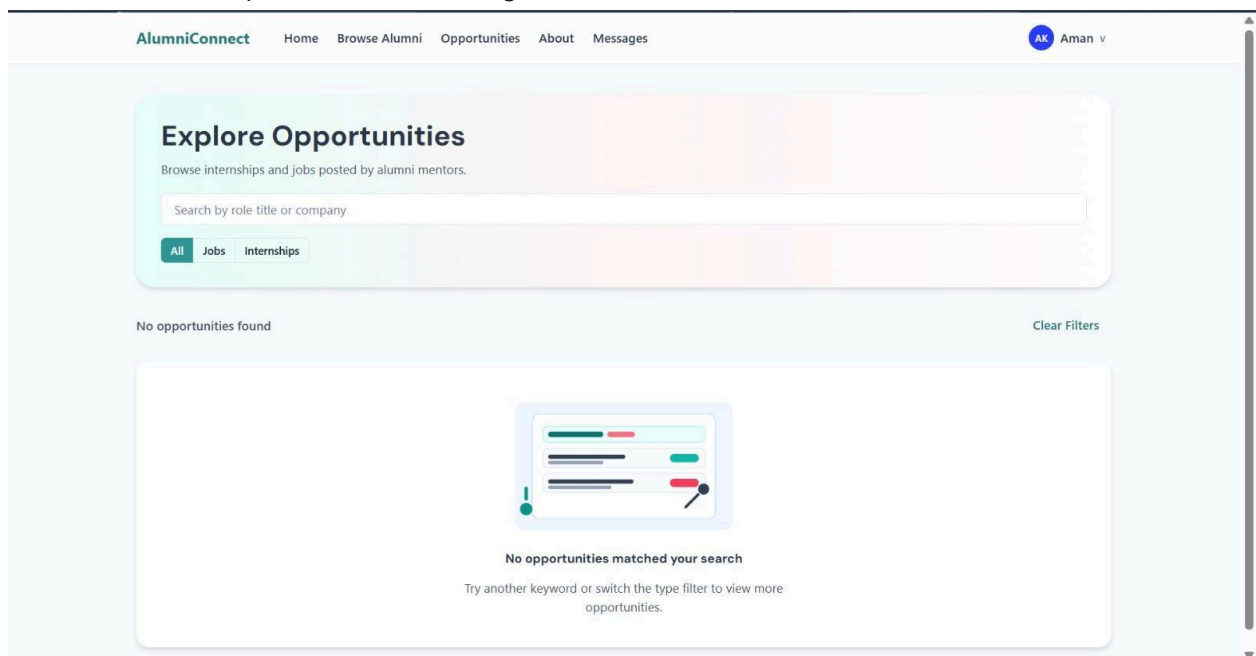
- Send and track mentorship requests
- Accept or reject requests

- Communicate with mentors and mentees



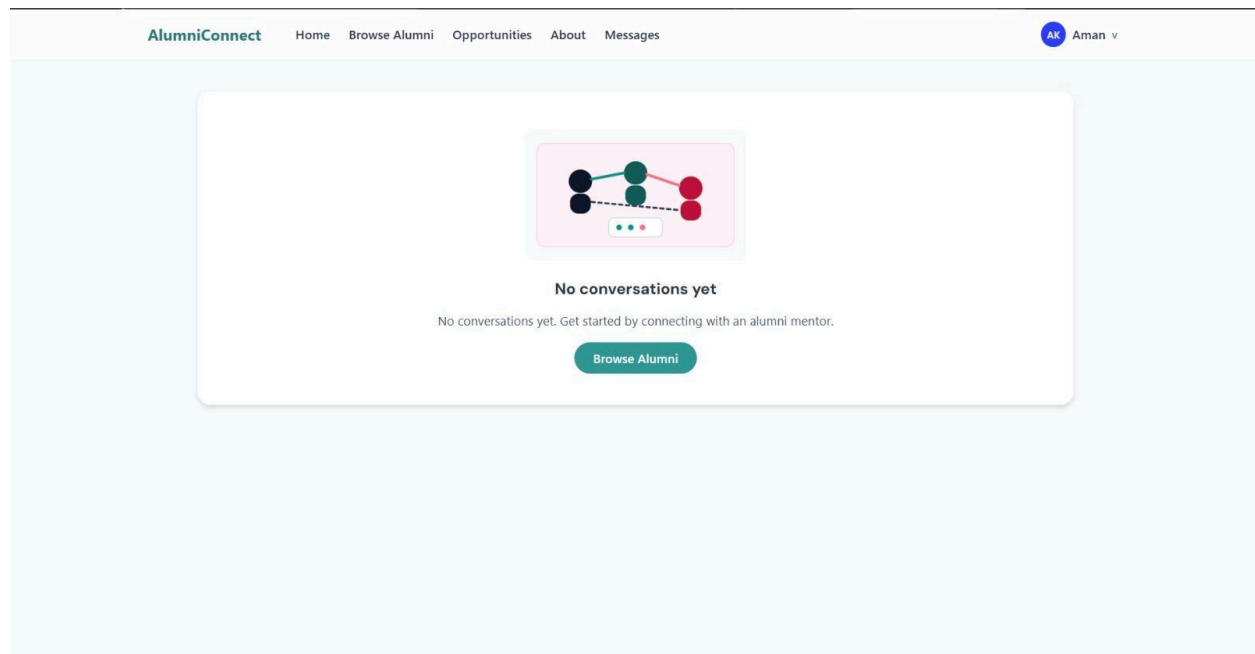
Opportunity Sharing

- Post job openings and internship opportunities
- Browse and explore available listings



Messaging

- Built-in messaging system for student-alumni communication
- Request-based connection before messaging



19. Optional Advanced Features

- Skill-based mentor recommendation using AI
- Real-time chat system using WebSockets
- Video mentorship sessions integration
- Alumni events management
- Professional networking groups
- AI-based career guidance and roadmap generation
- Push notifications for mentorship updates

20. Learning Outcomes

By completing this project, the team gained the following learnings:

- Full-stack MERN application development
- Professional networking platform design
- Role-based authentication and authorization
- Messaging and mentorship system implementation
- REST API development and testing

- Database schema design with MongoDB and Mongoose
- Scalable web application architecture
- Team collaboration using Git and GitHub
- UI/UX design for multi-role platforms

Links to the video explanations of the code and the project:

- Front-End Explanation video:
<https://drive.google.com/file/d/15KzxOV2shvTAoNSK2Q6zWdEVnJ-pjypt/view?usp=sharing>
- Back-End Explanation video:
https://drive.google.com/file/d/1_Qr2emw6WIPpNax8utSzCJuAWhILcbd0/view?usp=sharing
- Project Overview video:
https://drive.google.com/file/d/1WYyOzUPD_H4BNR8yqxyg1lekF_xzxrMV/view?usp=sharing